

Unboxing the Black Box: Quantitative Analysis of Physical Fragmentation in Modern SSDs

Abstract

Conventional flash-based SSDs suffer from severe performance degradation as storage aging accumulates logical and physical fragmentation. However, reproducing realistic aged states for evaluation remains prohibitively time-consuming and inherently non-deterministic due to stochastic background FTL operations, while internal physical fragmentation remains an unquantifiable black box. We present a high-fidelity, snapshot-based SSD emulation framework that captures the comprehensive internal structural layout of an aged device and instantly restores it, enabling perfectly deterministic and reproducible evaluation without replaying protracted live aging workloads. Built on this infrastructure, our framework goes beyond traditional evaluation paradigms by architecturally decoupling the storage stack to introduce a controllable physical fragmentation model that systematically isolates, modulates, and contrasts the distinct impacts of both logical and physical fragmentation. Our quantitative evaluation demonstrates that physical fragmentation severely diminishes hardware throughput and degrades sequential read performance, particularly when intricate internal device parallelism becomes saturated. Also, the proposed framework provides a rigorous, practical foundation for studying storage decay, FTL behavior, and multi-dimensional performance sensitivity under highly controlled, independently tunable conditions.

1 Introduction

Beyond conventional SSDs, novel storage architectures, including Zoned Namespace (ZNS) SSD, Open Channel SSD (OCSSD), Key-Value SSD (KVSSD), have been proposed to optimize performance and storage efficiency. More recently, emerging technologies such as Flexible Data Placement (FDP) and Compute eXpress Link (CXL)-attached storage are gaining attention as they enable smarter, more capable storage devices. This diversification of device types has expanded the storage device design space, prompting extensive research efforts to explore novel device paradigms.

Although these novel architectures demonstrate significant potential and promise performance gains, their deployment is still prone to long-term performance degradation due to *aging* and/or *fragmentation*. The aging, the situation where data intended to be contiguous in an address space is instead scattered across non-contiguous locations, can be induced under several scenarios. For example, repeated creation and deletion of varying-sized files fragment free space, forcing subsequent data writes to be split across non-contiguous logical block addresses. Concurrently, at the device layer,

multi-stream concurrent write traffic causes independent data payloads to intermix at the host interface, destroying spatial locality on the underlying physical flash channels. Furthermore, internal, asynchronous FTL management tasks, such as garbage collection and data scrubbing, exacerbate this physical disorder while relocating data.

This fragmentation is known to be a significant performance bottleneck in modern storage stacks. As a device and filesystem age, the layout of data becomes increasingly complicated. However, standard host-side I/O interfaces are fundamentally designed on the principle that a single command processes one contiguous address range. Therefore, this multi-dimensional structural degradation can severely diminish hardware throughput.

Consequently, accurately "aging" a storage device to a steady state is an essential prerequisite for systems research, ensuring that performance evaluations reflect realistic, long-term deployment conditions regardless of the specific architecture under review. Several tools are available for aging storage devices [1, 9, 19, 34]. However, collecting live system data during the aging process is time-consuming, and since the traces are quite large, the duration required for aging is significantly extended [30]. Existing aging tools can require days of execution before reaching meaningful fragmentation. For example, our evaluation using a 2 TB device with Geriatrix [19] required approximately 13 days to generate a properly aged device instance. This implies that a single evaluation run requires at least 13 days of aging time, which is prohibitively long. Moreover, even if we apply the same operations to age the device, the non-deterministic internal operations can make the physical data placement completely different from run to run [11]. Thus, it is almost impossible to ensure identical baseline conditions across multiple iterations of performance evaluation.

Meanwhile, physical fragmentation has emerged as a dominant bottleneck in modern storage stacks due to the intricate internal parallelism (e.g., channels, packages, and dies) that flash devices exploit for peak performance [18]. Mapping data to suboptimal physical locations forces serialized access and drives severe performance degradation. As modern storage device performance continues to improve, the overhead associated with this interfacing and serialization is no longer negligible. However, because this physical fragmentation is deeply buried within proprietary FTLs and closed internal device architectures, its exact architectural implications have been largely unexplored.

This paper proposes a methodology that leverages a snapshot of the aged storage state. By leveraging a software-defined high-fidelity emulator, we can generate a snapshot that captures the precise internal state of modern SSDs. Using this snapshot, users can conduct reproducible and swift experiments by directly loading a pre-aged snapshot. This obviates the need to spend days reproducing structural aging via live real-world workloads. Furthermore, using the emulator enables us to directly observe and manipulate the placement of data within parallel architectures of modern SSDs. This capability provides unique opportunities to quantitatively observe the state of physical fragmentation that was hidden behind the flash translation layer (FTL) and to directly assess the implication of physical fragmentation on performance.

The contributions of this paper are as follows:

- The design and implementation of a high-fidelity SSD emulation framework that enables direct control, structural manipulation, and microscopic measurement of physical fragmentation hidden beneath the FTL.
- A multi-dimensional, decoupled quantitative analysis that goes beyond traditional evaluation paradigms to isolate, contrast, and map the independent and compounding performance impacts of both logical and physical fragmentation across the storage stack.
- An instant, snapshot-based aging infrastructure for SSD emulation that guarantees perfectly deterministic and reproducible experimental states while reducing setup overhead from days to seconds.

2 Background

2.1 Fragmentation

Fragmentation in computer systems refers to the condition in which free space or data is scattered across non-contiguous locations. Depending on the entity being fragmented, fragmentation can be categorized into *free space fragmentation* and *data fragmentation*.

Free space fragmentation can be further classified into *internal* and *external* fragmentation [3, 28]. Internal fragmentation occurs when a system employs fixed-size allocation policies, leading to allocated regions containing unused space. In contrast, external fragmentation occurs when a system repeatedly allocates and deallocates variable-sized chunks. Over time, these operations divide free space into small non-contiguous fragments scattered across the address space. This scattered free space impedes the allocation of large contiguous blocks, even when the total available free space is sufficient to satisfy the request.

Similar to free space, data can be fragmented as well. In the context of filesystems, data fragmentation occurs when a file is split into multiple chunks and stored in non-contiguous locations in a storage device. Notably, data fragmentation can occur even when enough contiguous space is available

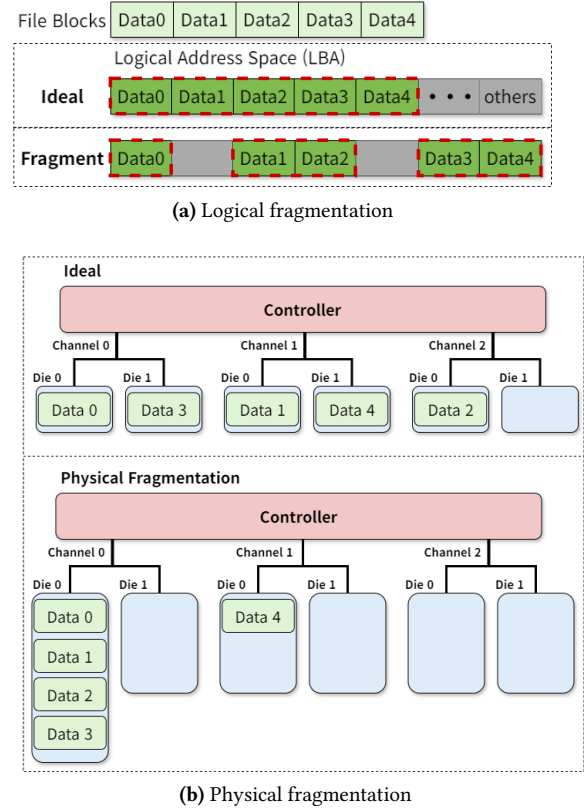


Figure 1. Types of fragmentation

to store the entire file, often due to prior fragmentation or allocation policies.

As illustrated in Figure 1, data fragmentation can happen in two forms: *logical fragmentation* and *physical fragmentation* [12]. Logical fragmentation refers to the condition in which data is split within the logical block address space. Specifically, modern storage devices expose their storage space through logical block addresses (LBAs), and the host specifies the target data for operations using a starting logical block number (LBN) along with the number of consecutive logical blocks. Consequently, filesystems must issue one I/O request per contiguous LBA chunk. When a file is logically fragmented, the filesystem must generate more I/O requests than it would for unfragmented data, thereby incurring increased interfacing overhead. Logical fragmentation occurs primarily due to the space allocation policies of filesystems [29].

Physical fragmentation originates from the discrepancy between logical and physical address spaces. Modern storage devices often utilize high-density storage media with complex characteristics and inherent limitations. Additionally, they typically employ sophisticated data paths and highly parallel architectures to deliver high performance. As a result, the logical address space exposed to the host does not

necessarily align with the physical address space on the storage media and its underlying architecture. Consequently, even if the host issues a request for a logically contiguous chunk, the device may process it through multiple internal operations.

In the era of HDDs, logical fragmentation was the primary source of performance degradation. The mechanical nature of hard drives dictates that fragmented files incur significant overhead due to seek time and rotational delay. In contrast, flash memory-based SSDs have no moving parts, eliminating both seek time and rotational delay. As a result, early research generally assumed that logical fragmentation had little to no impact on SSDs. Moreover, defragmentation was even considered harmful in SSDs, as it generates additional data writes that reduce the lifespan of the flash memory [12, 17, 26, 33]. However, more recent studies have discovered that filesystem aging and the accompanying fragmentation can indeed degrade SSD performance due to both logical and physical fragmentation [8, 15, 16, 18, 22, 24]. For instance, in modern SSDs with complex NAND flash architectures, data may be distributed unevenly across multiple channels and dies. This imbalance can lead to skewed workloads and reduced I/O parallelism, resulting in suboptimal performance. Furthermore, physical fragmentation is becoming increasingly pronounced in modern devices, where interfacing overhead constitutes a non-negligible portion of overall performance.

2.2 Related Work

Many studies have been carried out to investigate the characteristics of fragmentation and its implications for performance [8, 10, 15, 16]. To address the problems caused by fragmentation, two approaches have been investigated: (1) proactively prevent fragmentation from occurring [17, 18] and (2) reactively defragment already-fragmented data or free space in efficient ways [12, 24, 26, 33].

FragPicker [24] is a recent study that revisits fragmentation issues in the context of modern storage systems. The authors demonstrate that fragmentation can indeed lead to performance degradation, even on SSDs. To address this, FragPicker proposes a mechanism to identify hot files efficiently by monitoring read and write operations at VFS-level through eBPF.

Wang et al. [31] investigate how filesystems that employ out-of-place update schemes are prone to fragmentation. Specifically, in log-structured filesystems, update operations relocate data and it can lead to data fragmentation. The study specifically examines F2FS fragmentation on Android mobile devices. Additionally, the authors propose a scheme to leverage the file-based optimization (FBO) in UFS 4.0 [14] to mitigate data fragmentation.

However, aging filesystems and storage devices is a time-consuming process, often taking days or even longer. For instance, Smith et al. [30] reports the results of a 3-year study.

Such long-term processings are usually not feasible in many scenarios. Accordingly, there has been work attempted to control the preconditioning of filesystems and storage devices. Earlier studies primarily focused on data fragmentation, where data is scattered across the device. These studies, however, often neglect the impact of free space fragmentation and its broader implications on filesystem behavior and device aging.

Geriatix [19] is an SSD aging tool that improves upon previous approaches. It provides a more comprehensive and representative environment for evaluating SSD performance under aged conditions. Geriatix addresses earlier limitations by employing a set of built-in aging profiles, which control various parameters such as file size distributions and directory tree depths. These parameters influence the layout of the filesystem. The profiles guide the creation and deletion of files in patterns that mimic realistic workloads, thereby effectively reproducing fragmentation characteristics that naturally develop over prolonged usage.

Recent studies focus on an aspect other than free space fragmentation. Jun et al. [18] highlighted physical fragmentation. This work demonstrated that the performance issues caused by fragmentation stem not from request splitting but from die-level collisions occurring within the SSD. However, the FTL’s black-box design prevented direct measurement.

3 Motivation

It has been well-known that storage systems used in real-world workloads often experience significant performance degradation due to substantial fragmentation [8, 9, 15, 16, 19, 24, 25]. While this topic is not new, the nature of fragmentation and its impact have grown more complicated with the evolution of storage technologies. In the past, the implications of fragmentation were relatively straightforward. For instance, HDDs are fundamentally composed of mechanical and electromagnetic components. Although they rely on the LBA scheme, which is supposed to hide the actual data placement within the devices, the LBA space is tightly coupled with physical geometry in practice. This leads to closely aligned layouts between logical and physical address spaces, making *logical fragmentation* the primary performance concern. In contrast, SSDs operate entirely on electrical principles. Highly complicated internal architectures of SSDs often result in a logical placement of data in LBA space that bears little to no correlation with its actual physical location within the device. This *decoupling* introduces a greater concern of *physical fragmentation*, which is becoming an increasing concern in modern storage systems [13, 18, 23, 31, 32]. Moreover, due to the high performance of modern storage devices, the I/O interfacing overhead becomes increasingly significant, making the logical fragmentation still impactful.

In this sense, we argue that fragmentation presents complicated performance implications in modern storage systems and must be considered from both logical and physical dimensions. However, existing studies often disregard the combined effects of these two types of fragmentations.

First, preconditioning storage systems demands a significant amount of time and effort, which is often infeasible. Given the substantial impact of fragmentation, it is highly desirable to evaluate performance on aged systems. Several tools have been proposed to facilitate controlled and realistic aging [1, 9, 19, 30]. However, this process typically requires at least several hours, as we present in Section 6. In real-world systems, aging occurs over several weeks, months, or even years [30]. Therefore, many studies resort to evaluating systems on a fresh system condition, failing to properly incorporate the effects of fragmentation.

Second, prior preconditioning techniques are unable to control physical fragmentation. In HDDs, the closely aligned layouts between the logical and physical address spaces imply little undiscovered physical fragmentation. However, the sophisticated architectures of modern SSDs decouple these spaces, increasing the likelihood of physical fragmentation. To make matters worse, controlling the placement of data in SSDs is highly limited, as the internals of FTL is neither accessible nor modifiable in practice. Therefore, there is no explicit way to induce and/or control a desired degree of physical fragmentation with realistic workloads. This leaves physical fragmentation relatively underexplored in the existing literature. Indeed, to the best of the authors' knowledge, no tool currently exists that can perform aging while controlling physical fragmentation.

Finally, the growing diversity of storage devices complicates aging procedures. Recently, many novel device types have been introduced to meet the demand for high-performance and efficient data storage, and they often employ specialized I/O protocols tailored to their target workloads [5, 6, 27]. For instance, key-value SSDs and in-storage processing devices communicate with the host using non-traditional interfaces that do not follow the traditional LBA scheme. Others, such as ZNS SSDs and FDP SSDs, require the host to issue requests under strict operational constraints. This increasing variety in device interfaces make existing tools designed for conventional block devices with filesystems insufficient or even inapplicable.

These observations motivated us to explore a different approach to facilitate the research regarding storage device aging and fragmentation. We started with a simple idea of extracting the complete device-internal state, along with user data, from an aged SSD and reloading this *snapshot* into another SSD. This method allows the entire aged state to be effectively replicated without time-consuming, effort-intensive processes of traditional workload-based aging. We realized this concept using a storage emulator. In addition, we extended the emulator to explore various aspects of physical

fragmentation and analyze their impact on storage performance.

4 Device Snapshotting using an Emulator

To achieve the goals of this study, it is crucial to use a high-fidelity emulator capable of modeling the sophisticated internals of modern SSDs. We built our idea upon NVMeVirt [20, 21]. Unlike other emulators, NVMeVirt emulates devices by processing NVMe commands at the PCIe level. This level of emulation provides a flexible and realistic environment for evaluating various device types¹. NVMeVirt employs a realistic data processing model that covers the full hierarchy of storage hardware components, from PCIe links down to channels, dies, blocks, and pages. It accurately models the inherent parallelism of modern SSDs by supporting multiple FTL instances within an emulated device, which enables independent data processing partitioned by LBA ranges. Furthermore, it incorporates a buffered I/O model where inbound data transferred over the PCIe link is first staged in a write buffer before being dispatched to an appropriate FTL instance. These capabilities make NVMeVirt an ideal foundation for our research.

Based on the vanilla implementation of NVMeVirt, we implemented the device snapshotting feature. First, we extended the original emulator implementation to support the device snapshot feature. Specifically, we extended the sysfs interface of NVMeVirt to make a snapshot with the current internal states or to initialize a new instance with an existing snapshot. For making a complete snapshot, we extract four categories of data from the emulated device instance. First, we capture the entire mapping table used by the FTL for address translation and data placement. Second, we gather block management structures, including free block lists, pool block lists, and victim block queues. These are used for tracking SSD block status and allocating blocks for new writes. Third, we save the device configurations, such as the number of channels, the size of block and page, and the operation timing parameters. Finally, we obtain the entire contents of the storage, represented as a linear block in memory. Collectively, these data sources enable us to faithfully replicate and analyze the exact status of an aged SSD. They are stored in a single file to facilitate distribution and reproduction of emulation instances.

Restoring the snapshot takes place in a reversed way. NVMeVirt first loads the LBA space with the stored storage contents. Then, its internal states are restored from the snapshot file. During the restoration, the device is in a detached state so that no concurrent access to the device occurs. After the restoration, the device is reconnected to the host PCIe bus.

¹NVMeVirt currently supports conventional SSDs, NVM SSDs, key-value SSDs, and ZNS SSDs, with support for FDP SSDs in progress.

Our initial prototype discovered that most of the snapshot saving and restoration take for dumping and loading storage contents. For efficient storage of storage contents within the snapshot, we optimized the prototype to store the valid data only. The mapping table stores only the contents of valid LPNs, thereby omitting unallocated or invalid regions. This approach significantly reduces both snapshot file size and processing time.

5 Taming Physical Fragmentation

5.1 Challenges in Taming Physical Fragmentation

Modern SSDs achieve high I/O performance by leveraging internal parallelism across multiple channels and dies. Consequently, a non-uniform distribution of data across these components can disturb the parallelism, leading to performance degradation. However, prior studies regarding this topic have been limited by the "black-box" nature of proprietary FTL, which obstructs direct analysis of physical fragmentation and its implications on performance.

Our work overcomes these limitations by leveraging the high-fidelity emulator. Specifically, NVMeVirt provides direct access into modern SSD internals, including FTL mapping tables, block management structures, metadata, and the operational states of individual channels and blocks. This deep visibility into SSD internals enables a quantitative and detailed analysis of physical fragmentation, which was previously infeasible due to hardware-constrained setups. To further evaluate the performance impact of physical fragmentation, we designed and implemented a mechanism that allows users to quantitatively control the degree of physical fragmentation on a scale from 0 to 100%.

We acknowledge that the internal logic of commercial SSDs may diverge from the emulated models. However, our goal is not to perfectly replicate a specific proprietary drive. Rather, we utilize the emulator as a controlled environment to explore the fundamental architectural consequences of physical fragmentation and its impact on the modern storage stack.

5.2 Modeling Physical Fragmentation

A straightforward way to induce physical fragmentation would be to randomly relocate data across the device. However, such an approach has two major limitations. First, it creates a high-dimensional and unintuitive parameter space, making it difficult to control the degree of fragmentation precisely. Second, random relocation quickly converges toward a near-uniform distribution, which makes it hard to distinguish moderate fragmentation from severe fragmentation.

To overcome these limitations, we designed a new model based on three intuitive parameters.

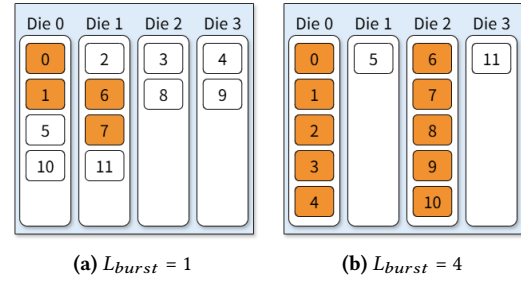


Figure 2. Impact of L_{burst} : Die collision at Data 0 and Data 6

Insight 1: collision probability. What we aimed for was a parameter that could determine how frequently the placement deviates from an ideal round-robin placement. This requirement motivated the introduction of P_{degree} : for each write, the model draws a random number $R \in [0, 100]$. If $R \leq P_{degree}$, the write is placed on the same die as the previous one, rather than on the next die (except when $P_{degree} = 0$). Consequently, when $P_{degree} = 0$, the model maintains a strict round-robin placement; conversely, when $P_{degree} = 100$, it directs all writes to a single die.

Insight 2: bursty allocation to create structural die collision. In addition to the frequency of physical fragmentation, we introduced the parameter L_{burst} to model the structural impact of individual events. While P_{degree} dictates the frequency of collision events, L_{burst} controls the structural severity of fragmentation events by defining the number of consecutive pages allocated to the same die upon a collision.

As shown in Figure 2a, when we set L_{burst} to a small value, the impact remains minor even if P_{degree} triggers a fragmentation event. Consequently, performance degrades relatively gradually as P_{degree} increases.

However, as illustrated in Figure 2b, a larger L_{burst} value causes even a single collision event to concentrate multiple pages onto a specific die. This imposes a severe die collision load on the targeted die, driving a steep decline in performance.

In real-world environments, SSDs frequently process interleaved data streams resulting from concurrent I/O requests. Since typical FTLs assign dies in a round-robin manner based on arrival order, concurrent writes from multiple processes can lead to physical fragmentation. As illustrated in Figure 3a, when no intervening writes occur between logically contiguous pages A and B, the FTL places them on different dies, facilitating parallel operations. In contrast, Figure 3b shows that intervening writes from concurrent streams displace A and B onto the same die, forcing serialized access and degraded read performance.

Insight 3: stream partitioning to model data interleaving. To model this data interleaving behavior, we introduce

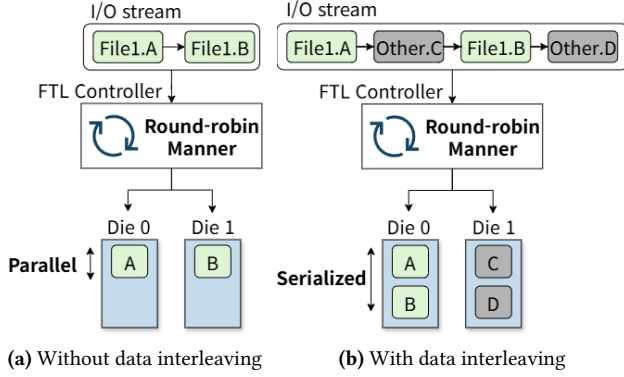


Figure 3. Why need data Interleaving mechanism

the parameter $N_{streams}$. The logical address space is partitioned into $N_{streams}$ equal-sized segments, each representing an independent file stream. The model cycles through these segments in round-robin order, interleaving pages from different streams at the FTL level. When $N_{streams} = 1$, all writes are handled sequentially without interleaving. As $N_{streams}$ increases, writes from different segments become increasingly intermixed, replicating the inter-stream interference.

5.3 Stochastic Physical Fragmentation (SPF) Model

In conclusion, P_{degree} controls the frequency of die-collision events, L_{burst} the structural severity of each event, and $N_{streams}$ the degree of data interleaving. By integrating these parameters, we implemented the Stochastic Physical Fragmentation (SPF) model.

Figure 4 illustrates an example of the SPF model with $N_{streams} = 4$, $P_{degree} = 50$, and $L_{burst} = 3$. In this configuration, the logical address space is partitioned into four equal-sized chunks (A, B, C, D), each representing an independent file stream. The model cycles through these streams in a round-robin order, writing a fixed-sized data unit from each chunk in turn. This causes pages from different streams to be intermixed at the FTL level, replicating the physical fragmentation induced by concurrent I/O in real-world workloads.

For each write, the model draws a random value R from $[0, 100]$ and compares it against $P_{degree} = 50$. When $R > P_{degree}$ (e.g., event C1), the write proceeds under the round-robin policy, distributing data evenly across dies. When $R \leq P_{degree}$ (e.g., event C2), the model allocates $L_{burst} = 3$ consecutive pages to the same die, creating a fragmented cluster that structurally disrupts the otherwise uniform layout.

6 Evaluation

6.1 Evaluation Setup

We implemented the proposed device snapshotting and fragmentation analysis schemes based on NVMeVirt [21] on Linux kernel 5.10.0. Evaluations were performed on a server equipped with two Intel Xeon Gold 5215 CPUs (2.50 GHz, 10 cores per socket) and 384 GiB of DRAM distributed across two NUMA nodes. The NVMeVirt instances were configured to emulate conventional SSDs with capacities of 128 GiB and 64 GiB. All other settings followed the specifications provided in the original NVMeVirt paper.

To isolate the impact of logical and physical fragmentation, we tuned several operating system configurations. Specifically, we employed the `O_DIRECT` flag and flushed the page cache before each measurement to eliminate the influence of the OS-level cache. In addition, we disabled the kernel-level readahead feature to ensure that OS prefetching did not mask the underlying device latency. Unless otherwise specified, these configurations remained consistent across all experiments.

6.2 Impact of Host I/O Buffer Fragmentation

First, during our preliminary evaluations, we observed highly inconsistent run-to-run performance. Despite explicitly placing data in a severely fragmented/skewed layout, the measured performance varied significantly. In some instances, we could observe significant performance degradation due to the fragmentation, while in others, the penalty was negligible.

An in-depth analysis throughout the entire I/O stack revealed that NVMe commands were using long PRP (Physical Region Page) lists for payloads, which was unexpected. This fragmented payload was traced back to physically non-contiguous I/O buffers at the host level. By default, `fio` [4], the primary benchmarking tool used in our evaluation, allocates I/O buffers using the standard memory allocation mechanism (i.e., `malloc()`). Due to the first-touch memory allocation policy in Linux, the I/O buffers are populated with discrete 4 KiB basepages [3, 7]. As these basepages are allocated independently from free page lists, they are most likely to be physically non-contiguous. In our trace-level profiling, we observed that 99.99% (6,859,850 out of 6,859,852) of PRP entries generated under the default memory allocation were physically non-contiguous. Consequently, the payload for an NVMe command must be described by a list of 4 KiB basepages, thereby forcing the device to process the transfer in a page-by-page manner. This leads benchmark operations to be processed as if operating under severe fragmentation.

To eliminate this host-side interference, we configured `fio` with `-iomem=mmaphuge` option to allocate I/O buffers with physically contiguous hugepages. This adjustment drastically reduced the ratio of non-contiguous PRPs to 0.003% (221 out of 6,859,852). Consequently, we verified that the

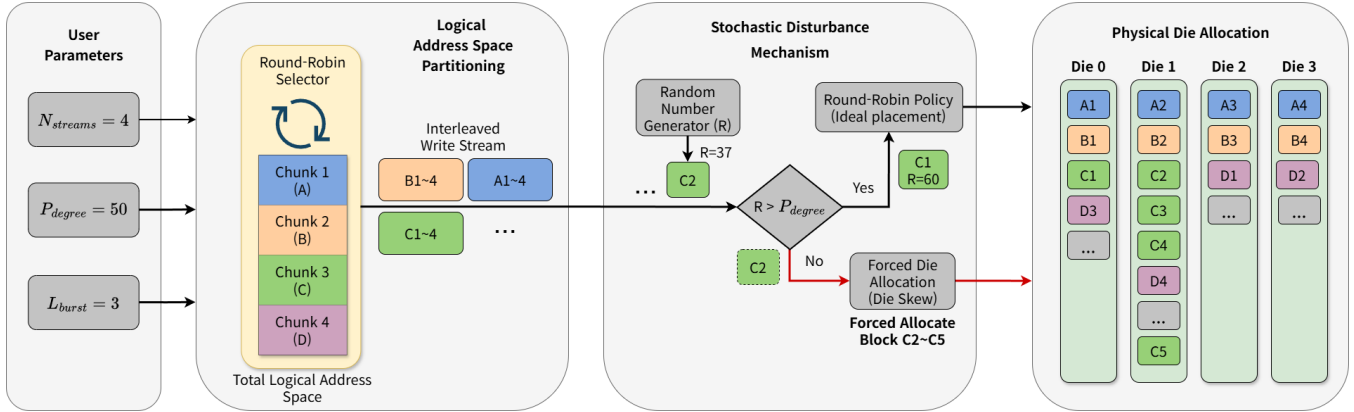


Figure 4. An operational trace with parameters $N_{streams} = 4$, $P_{degree} = 50$, and $L_{burst} = 3$.

physically contiguous payloads make the performance measurements consistent and reproducible.

6.3 Snapshotting

6.3.1 Temporal Cost of Aging. Appropriate aging of a storage system is time-consuming, especially considering the low probability of fragmentation across a large storage capacity. To quantify the temporal cost of the aging process, we conducted experiments on a 2 TiB Samsung 980 Pro SSD and a 512 GiB Samsung 970 Pro SSD. We performed the aging following the methodology introduced by Geriatrix [19]. Specifically, Geriatrix ages a file system by repeatedly creating and deleting files according to three real-world distribution profiles (i.e., file age, file size, and directory depth) derived from the study by Agrawal et al. [2]. It continues operations until the actual file system state converges to the target distributions. In our experiments, we configured Geriatrix with a target utilization of 80% using 16 threads.

Our measurements showed that reaching a steady state, where the SSD is sufficiently fragmented internally and its performance stabilizes at a lower bound, required approximately 13 days for the 2 TiB drive, and took around 25 hours even for the smaller 512 GiB SSD. These results confirm that aging does influence I/O performance, yet evaluating under properly aged conditions would be prohibitive due to the temporal burden and effort.

6.3.2 Snapshotting Performance. To improve snapshot efficiency, we adopted the valid-only dump scheme. To evaluate the effectiveness of the proposed method, we compared its performance against the full dump approach, which captures the entire space, using a 128 GiB emulated SSD while varying storage utilization to 20%, 50%, and 80%. As summarized in Table 1, the valid-only dump method reduced file size and latency during both the saving and loading phases compared to the full dump. This performance gain occurs because while the full dump unnecessarily saves data from invalid and unallocated regions with no logical significance,

Table 1. Snapshot performance comparison between valid-only dump and full dump under varying storage utilization (128 GiB emulated SSD).

Utilization	Method	Save (sec)	Load (sec)	Size (GiB)
20%	Valid-only	81.30	55.58	28
	Full Dump	172.98	117.15	130
50%	Valid-only	122.15	98.44	63
	Full Dump	172.24	116.95	130
80%	Valid-only	155.69	97.05	95
	Full Dump	174.63	117.24	130

the valid-only technique consumes resources in linear proportion to the actual amount of valid data stored.

6.4 Fragmentation Consistency

To determine whether the trend of aging in our emulation aligns with that of an actual SSD, we compared the performance trends of our emulator (64 GiB NVMeVirt) with a real SSD (2 TiB Samsung 980 Pro). We clarify that the purpose of this experiment is to verify that the emulator can reproduce similar behavior of aged devices, not to match absolute performance numbers. In particular, we investigate whether both systems show similar degradation patterns as fragmentation increases.

The real SSD was aged by repeatedly executing a database workload of approximately 11 GiB per round, following the aging methodology of Ji et al. [16]. The emulator was aged using the same method with a 1.1 GiB workload per round. We continuously aged both devices until their throughput degraded and converged to a steady lower bound. We then normalized the entire aging progression and directly compared their degradation trends at equivalent aging lifecycles.

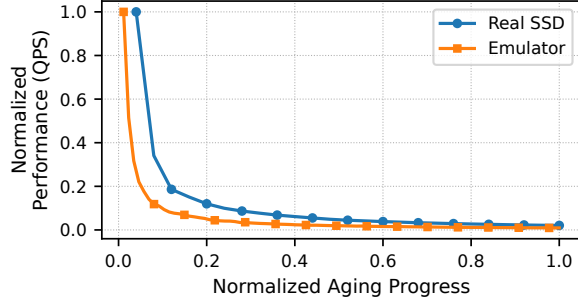


Figure 5. SELECT QPS across aging progress. Both environments show rapid initial degradation followed by monotonic decline.

After aging, we evaluated the performance through read-intensive database workloads. Specifically, we measured SELECT QPS, which represents the number of queries processed per second while repeatedly executing simple SELECT queries on an SQLite database for 30 seconds. These workloads exhibit I/O patterns that are sensitive to physical fragmentation due to their sequential scanning characteristics.

Figure 5 compares the SELECT QPS between the two environments. Due to the disparity in storage capacity and the resulting difference in the absolute number of aging rounds, we normalized the x-axis to the completion percentage of the total rounds for each environment. Both environments exhibit similar trends: a rapid initial degradation in performance followed by a persistent monotonic decline. In the real SSD, the throughput plummeted from 284 QPS to 43 QPS within the first 28 rounds, after which it monotonically decreased to 5.89 QPS by the 175th round, ultimately resulting in a 97.9% performance degradation from the baseline. The emulator successfully replicates the two-stage pattern, with throughput decreasing from an initial 22,844 QPS to 210 QPS (a 99.1% reduction) by the end of 87 rounds.

While the previous experiment confirmed the consistency of macroscopic aging trends, we further investigated whether the emulator could capture the impact of physical fragmentation on various performance metrics. For this micro-level analysis, we utilized a 512 GiB Samsung 970 Pro as the baseline. Initially, we measured performance under a clean condition using Filebench `video_server` and `fio`. Then, we performed aging using the Geriatrix framework. At the same time, we generated random overwrites to the files created for the benchmarks. These overwrites led the FTL to perform out-of-place updates, which scattered the physical data layout of the benchmark files. We then measured the performance under the aged condition. After the measurement, we attempted to eliminate physical fragmentation; we dumped the contents of the storage and sequentially wrote them back using `dd`. Then, we measured the performance again under the recovered condition. This way, we strictly controlled the

number of extents—an indicator of logical fragmentation—consequently isolating physical fragmentation as an independent variable.

As summarized in Table 2, measurements from the real SSD reveal that in the aged state, the bandwidths for Filebench and `fio` decreased from 1,534 MB/s to 1,331 MB/s and from 1,633 MB/s to 1,448 MB/s, respectively. Notably, the tail latency for `fio` surged, with p99.9 increasing from 322 μ s to 627 μ s and p99.99 from 396 μ s to 873 μ s. However, following the `dd-rewrite`, which perfectly rectifies the physical layout across NAND channels, both bandwidth and tail latency fully recovered to their initial conditions. This confirms that the observed performance degradation is entirely attributable to physical fragmentation within the FTL.

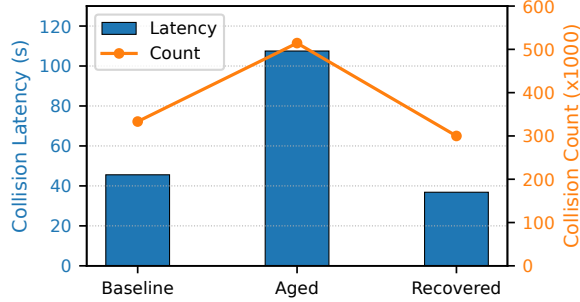
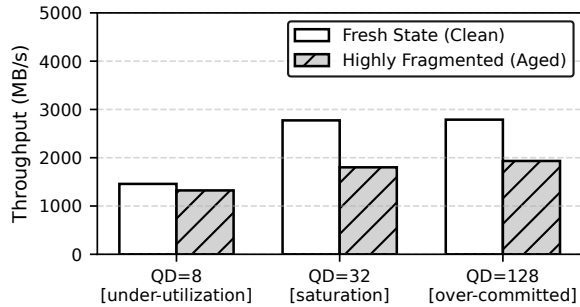
The emulator successfully replicated these trends. Under the emulator’s aged state, the Filebench bandwidth decreased from 3,349 MB/s to 2,933 MB/s, and the `fio` bandwidth dropped from 3,017 MB/s to 2,020 MB/s. The p99.9 latency for `fio` also surged similarly to the real SSD, demonstrating highly consistent overall performance degradation trends. However, in the emulator environment, the bandwidth for `fio` exhibited a much steeper drop compared to Filebench. Although the specific causes for this discrepancy will be analyzed in-depth later, at present, we attribute this difference in degradation rates to sophisticated performance optimizations in commercial SSD products, such as prefetching and buffering.

Furthermore, the white-box nature of the emulator enables direct observation and measurement of internal die collision behaviors. To quantify the hardware penalty incurred by physical fragmentation, we introduced two internal metrics: *die collision latency* and *die collision count*. Die collision latency refers to the total accumulated wait time of I/O requests delayed by a busy die, whereas die collision count represents the total occurrences where I/O requests were forced to wait due to such die contention. These collision metrics spike when concurrent read requests fail to distribute evenly across multiple dies and instead concentrate on a specific die. Therefore, high die collision counts and latencies demonstrate that subsequent I/O requests cannot be processed in parallel across multiple dies, but are instead serialized and sequentially backlogged on a single die.

As illustrated in Figure 6, under the `fio` workload, the aged state exhibits an increase in die collision count from 333K to 514K and a more than twofold explosion in die collision latency compared to the initial baseline, directly demonstrating severe physical resource contention. In contrast, following the `dd` overwrite, both die collision latency and count were restored, even dropping slightly below the initial baseline levels. This is because the initial baseline state inherently includes a certain degree of collisions caused by the overlapping of asynchronous I/O requests induced by the configured `iodepth` during the file creation process. These changes in internal metrics indicate that physical fragmentation directly

Table 2. Both environments maintain constant extents, isolating the impact of physical fragmentation.

Device	Workload / Metric	Initial	Aged	Recovered
Physical SSD (Samsung 970 Pro)	Filebench BW	1,535 MB/s	1,331 MB/s	1,535 MB/s
	fio Seq Read BW	1,633 MB/s	1,448 MB/s	1,634 MB/s
	fio 99.9% / 99.99% Tail Latency	322 μ s / 396 μ s	627 μ s / 873 μ s	318 μ s / 396 μ s
Emulator (NVMeVirt)	Filebench BW	3,349 MB/s	2,933 MB/s	3,349.2 MB/s
	fio Seq Read BW	3,017 MB/s	2,020 MB/s	3,020 MB/s
	fio 99.9% / 99.99% Tail Latency	586 μ s / 619 μ s	1045 μ s / 1123 μ s	635 μ s / 693 μ s

**Figure 6.** Internal die collision metrics under fio workload across different aging states.**Figure 7.** Throughput degradation across queue depths

dictates parallel processing efficiency at the NAND level, demonstrating that the employed emulator effectively captures the operational trends of the storage.

6.5 Sensitivity Analysis

In this section, we investigate the correlation between SSD read parameters and physical fragmentation using an SSD emulator. The emulated instance is configured to have 16 dies (8 channels $\times$ 2 ways) and a 32 KiB page size.

6.5.1 Impact of Queue Depth. It is often intuitively expected that the impact of fragmentation increases linearly as the queue depth (QD) of commands grows deeper. However, we found counter-intuitive results. To identify the root cause of this behavior, we analyzed both the saturation of physical resources and the total queueing time.

First, during the under-utilization condition (QD=8), where system resources remain idle. Because the number of concurrent requests is fewer than the physical dies (16 die), the probability of die-level contention is low. Our measurements indicate that the average die active ratio reached approximately 45% (Figure 8a), while the total queueing time remained at a relatively marginal level, serving as the baseline (1x) when considering the explosive surge observed under deeper queue conditions. Even if fragmentation-induced delays occur, the remaining idle dies continue to service other pending requests without contention, preventing pipeline stalls and limiting the overall throughput degradation marginal. (see Figure 7).

Second, the situation reverses upon reaching a saturation point (QD=32), where all dies operate at maximum efficiency. At this point, the average die active ratio sharply increases to roughly 90%, indicating that all dies are operating at maximum efficiency without any slack. Consequently, queueing time begins to increase. Because every resource is utilized to its full capacity, even minor die collisions caused by fragmentation immediately trigger bottlenecks that stall the entire pipeline. As a result, the performance degradation reaches its peak. This suggests that the system becomes most vulnerable to physical discontinuity precisely at the moment it achieves maximum efficiency.

Finally, in the over-committed condition (QD=128), excessive queue depth keeps all dies in a constant "busy" state, leading to full saturation. The die active ratio saturates at approximately 91% (a level similar to QD=32), meaning that the physical processing capacity has reached its limit. However, as illustrated in Figure 8b, the total queueing time increases rapidly. While die collisions still occur, their impact becomes limited. Because each die maintains a sufficiently deep queue, the system immediately executes the next pending request even if a specific task is delayed, thereby filling the idle gaps in the pipeline. In this context, the overflowing queue acts as a buffer that masks individual delays caused by fragmentation, which relatively mitigates the degradation of throughput.

These results indicate that the performance impact of physical fragmentation is non-linear with respect to queue depth: the system is most vulnerable precisely when it operates

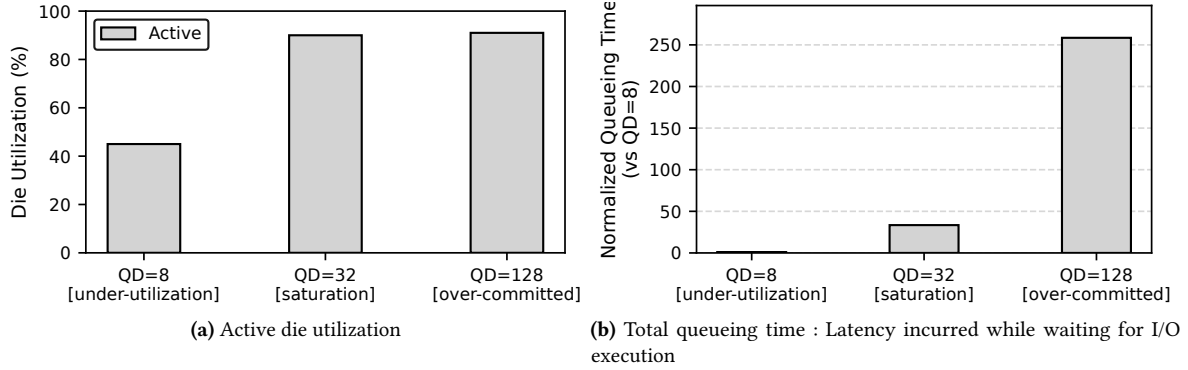


Figure 8. Saturation levels relative to queue depth

at peak efficiency, while over-saturation paradoxically mitigates the impact by turning the deep queue into a collision buffer.

6.6 Microbenchmark: Physical Fragmentation Modeling

In this experiment, we evaluate the behavior and efficacy of the parameters introduced in our proposed SPF model. Specifically, we aim to verify how the three parameters (P_{degree} , L_{burst} , $N_{streams}$) independently control internal die collisions and performance degradation. By adjusting each parameter individually, we empirically demonstrate how they effectively control the severity and patterns of physical resource contention within the emulator.

To achieve this without the interference of internal logical fragmentation, we precreated files sequentially and measured the sequential read performance over these files using `fio`. This setup allowed us to evaluate the purely physical fragmentation of our proposed parameters.

P_{degree} : Controlling the Frequency of Physical Fragmentation As shown in Figure 9, the bandwidth consistently decreases while the die collision latency increases as P_{degree} grows. Although the magnitude of the performance drop varies depending on the settings of other parameters such as L_{burst} and $N_{streams}$, the overall downward trend in bandwidth remains consistent across all conditions as P_{degree} increases. This behavior occurs because, in lower P_{degree} ranges, the model distributes data relatively evenly across multiple dies, thereby preserving the internal parallelism of the SSD. Conversely, increasing P_{degree} intensifies the concentration of I/O requests on specific dies. Particularly in high P_{degree} ranges, this concentration severely weakens internal parallelism, leading to a significant drop in bandwidth and a surge in latency. Consequently, P_{degree} serves as the most dominant variable in determining the performance degradation trends caused by physical fragmentation.

L_{burst} : Controlling the Structural Severity of Die Collision Under the condition shown in Figure 10, the bandwidth

decreases as L_{burst} increases from 1 to 64. This decline results from the fact that larger L_{burst} values extend the duration of die collisions, thereby exacerbating the bottleneck. In contrast, under conditions with already severe fragmentation ($P_{degree} = 90$), the additional bandwidth drop caused by increasing L_{burst} is relatively smaller. This phenomenon occurs because the majority of data is already skewed toward specific dies, establishing extremely long die collisions as a baseline. Nevertheless, we consistently observe the overall downward trend in bandwidth, confirming that L_{burst} plays an independent role in increasing the structural severity of existing die collisions.

$N_{streams}$: Controlling the Data Interleaving $N_{streams}$ is a parameter that simulates multiple data streams being written simultaneously. As shown in Figure 11, when $N_{streams}$ increases while fixing the collision frequency and length, data from different streams becomes increasingly intermixed. This intensifies FTL-level interleaving and scatters the physical placement, which weakens internal parallelism. However, the magnitude of this performance degradation is relatively gradual compared to P_{degree} or L_{burst} . Consequently, it is appropriate to view $N_{streams}$ as a secondary parameter for realistically replicating mutual interference in concurrent workloads rather than a variable that directly induces extreme bottlenecks.

6.7 Isolation of Logical and Physical Fragmentation

6.7.1 Experimental Setup. In real workloads, aging causes logical and physical fragmentation simultaneously. Thus, their respective penalties on I/O performance are often combined, and analyses have focused predominantly on the logical fragmentation aspect. Accordingly, we attempted to decouple these two factors to quantify their impacts independently. By doing so, we demonstrate that a comprehensive understanding of storage performance degradation requires evaluating both dimensions simultaneously, rather than focusing on either in isolation.

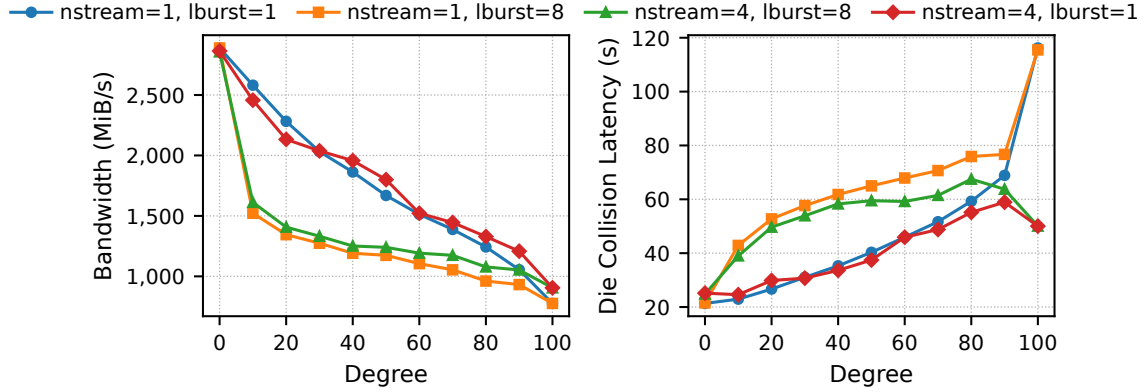


Figure 9. Performance impact of parameters on bandwidth and die collision latency

Table 3. Sequential and random read throughput, along with p99 latency, across the 2×2 fragmentation matrix.

Case	Description	Logical Frag	Physical Frag	Seq BW (MB/s)	Die collision Latency (s)	Rand BW (MB/s)
A	Logical only	✓	×	2803	155	1122
B	Physical only	×	✓	1690	186	1131
C	Both	✓	✓	1662	209	1113
D	Baseline	×	×	2930	162	1142

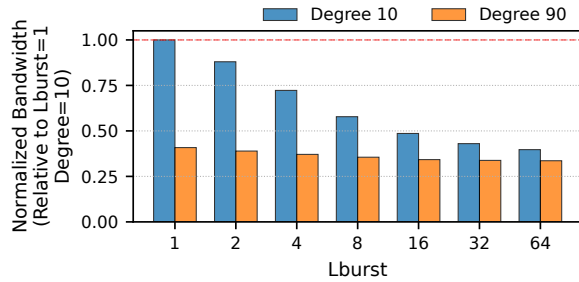


Figure 10. Impact of L_{burst} on Normalized Bandwidth

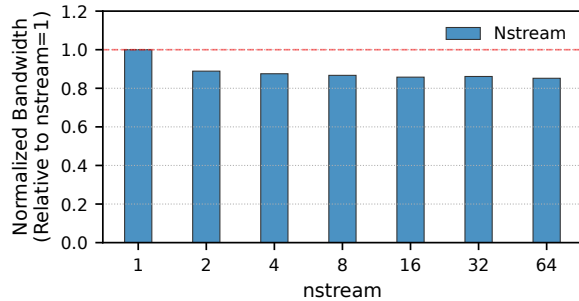


Figure 11. Impact of $N_{streams}$ on Normalized Bandwidth

To decouple and analyze the effects of logical and physical fragmentation, we designed a 2×2 experimental matrix along two independent variables: (1) the presence of logical fragmentation and (2) the presence of physical fragmentation.

Table 3 summarizes the four cases, and they are configured as follows.

For Cases A and C, we induced logical fragmentation by performing SQLite-based aging on the emulated SSD. This process fragmented and scattered files across numerous extents, and we recorded the resulting file set as a reference file list. Conversely, Cases D and B reconstructed the identical file set by sequentially writing each file at once using the previously recorded list. This approach eliminates file-level logical fragmentation.

We controlled physical fragmentation independently of logical fragmentation. We set the P_{degree} value to either 0 (Cases D and A) or 99 (Cases B and C). For $P_{degree} = 0$, we fixed $N_{streams}$ at 1; for $P_{degree} = 99$, we fixed the additional parameters L_{burst} at 8 and $N_{streams}$ at 4.

We measured performance by executing sequential reads with the fio tool. Specifically, we applied $iodepth=32$ and $bs=32$ KiB, and enabled the direct I/O and hugepage options. Additionally, we verified the logical fragmentation status of each case by checking the number of extents per file using the filefrag utility.

6.7.2 Result. Our analysis of the average extents per file confirmed that Cases A and C recorded an average of 62.1 extents per file, whereas Cases D and B averaged only 4.7. This demonstrates that we successfully applied logical fragmentation only to specific cases as intended, thereby validating our experimental design.

Introducing physical fragmentation only without logical fragmentation (Case D $\rightarrow$ B) significantly decreased the

throughput from 2,930 MB/s to 1,690 MB/s. In addition, we confirmed that physical fragmentation intensified compared to the baseline, as the die collision latency increased from 162s to 186s.

Case C, which contains both types of fragmentation, as expected, exhibited the worst performance (1,662 MB/s), confirming the combined effects of logical and physical fragmentation.

Meanwhile, consistent with the findings of Jun et al. [18], the random read bandwidth ranged between 1,113 and 1,142 MB/s across all four cases, showing a marginal performance difference of less than 2.5%. This is because random I/O requests are inherently distributed across multiple dies with uniform probability, regardless of the underlying logical or physical continuity. This stands in contrast to the severe performance degradation we observed in sequential reads.

Furthermore, Case A (logical fragmentation only) exhibited a smaller performance degradation than Case B (physical fragmentation only), which also aligns with the results of prior research [18]. Case A recorded the lowest die collision latency among all experimental groups. This value is even lower than that of the baseline (Case D), which we attribute to the fact that Case A’s experimental configuration eliminated even the minor physical fragmentation elements that inevitably occur while generating the baseline condition.

These results validate the recent findings by Jun et al. [18], confirming that the adverse impact of physical fragmentation outweighs that of logical fragmentation, acting as the primary bottleneck in modern SSD environments. However, going beyond simply identifying the root cause, our isolated 2×2 analysis demonstrates that the two dimensions of fragmentation operate independently and their penalties conflate in real-world settings. This implies that conventional storage research and aging methodologies, which focus on inducing or isolating logical fragmentation, are fundamentally insufficient. We emphasize that to accurately evaluate the performance of modern SSDs, researchers must transition to a holistic approach that simultaneously considers and independently controls both logical and physical fragmentation.

7 Conclusion

This study quantitatively demonstrates the phenomenon of physical fragmentation in SSDs, which has long been obscured by black-box FTL architectures. By combining a fine-grained snapshot-based emulation framework with our Stochastic Physical Fragmentation (SPF) model, we enable modulation and observation of the internal data layout. Furthermore, the proposed snapshot-based aging infrastructure shifts the storage evaluation paradigm by reducing preconditioning overhead from days to seconds, while guaranteeing perfect determinism and reproducibility. In future work, we plan to extend this model to modern host-managed storage architectures, such as FDP and ZNS.

References

- [1] Nitin Agrawal, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2009. Generating realistic impressions for file-system benchmarking. In *Proceedings of the 7th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, San Francisco, CA.
- [2] Nitin Agrawal, William J. Bolosky, John R. Douceur, and Jacob R. Lorch. 2007. A Five-Year Study of File-System Metadata.
- [3] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. 2023. *Operating Systems: Three Easy Pieces* (1.10 ed.). Arpaci-Dusseau Books.
- [4] Jens Axboe. [n. d.]. Flexible I/O Tester. <https://github.com/axboe/fio>.
- [5] Matias Bjorling, Javier Gonzalez, and Philippe Bonnet. 2017. Light-NVM: The Linux Open-Channel SSD Subsystem. In *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, Santa Clara, CA, USA, 359–374.
- [6] Matias Bjorling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. 2021. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*. USENIX Association, Santa Clara, CA, 689–703.
- [7] Daniel Bovet and Marco Cesati. 2005. *Understanding the Linux Kernel*. Oreilly & Associates Inc.
- [8] Alex Conway, Ainesh Bakshi, Yizheng Jiao, William Jannen, Yang Zhan, Jun Yuan, Michael A. Bender, Rob Johnson, Bradley C. Kuszmaul, Donald E. Porter, and Martin Farach-Colton. 2017. File Systems Fated for Senescence? Nonsense, Says Science!. In *Proceedings of the 15th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, Santa Clara, CA, USA, 45–58.
- [9] Alex Conway, Ainesh Bakshi, Yizheng Jiao, Yang Zhan, Michael A. Bender, William Jannen, Rob Johnson, Bradley C. Kuszmaul, Donald E. Porter, Jun Yuan, et al. 2017. How to fragment your file system. *USENIX; login* (2017).
- [10] Alex Conway, Eric Knorr, Yizheng Jiao, Michael A. Bender, William Jannen, Rob Johnson, Donald Porter, and Martin Farach-Colton. 2019. Filesystem Aging: It’s more Usage than Fullness. In *Proceedings of the 11th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*. USENIX Association, Renton, WA.
- [11] Gabriel Haas, Bohyun Lee, Philippe Bonnet, and Viktor Leis. 2025. SSD-iq: Uncovering the Hidden Side of SSD Performance. *Proc. VLDB Endow.* (2025).
- [12] Sangwook Shane Hahn, Sungjin Lee, Cheng Ji, Li-Pin Chang, Inhyuk Yee, Liang Shi, Chun Jason Xue, and Jihong Kim. 2017. Improving File System Performance of Mobile Storage Systems Using a Decoupled Defragmenter. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*. USENIX Association, Santa Clara, CA, 759–771.
- [13] Weizhou Huang, Hepei Wu, Shuhan Bai, Jian Zhou, and Fei Wu. 2025. Fraggie: Reducing File Fragmentation on DRAM-less SSD. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [14] JEDEC. [n. d.]. UFS: (Universal Flash Storage). <https://www.jedec.org/standards-documents/focus/flash/universal-flash-storage-ufs>.
- [15] Cheng Ji, Li-Pin Chang, Sangwook Shane Hahn, Sungjin Lee, Riwei Pan, Liang Shi, Jihong Kim, and Chun Jason Xue. 2019. File Fragmentation in Mobile Devices: Measurement, Evaluation, and Treatment. *TMC* 18, 9 (2019), 2062–2076.
- [16] Cheng Ji, Li-Pin Chang, Liang Shi, Chao Wu, Qiao Li, and Chun Jason Xue. 2016. An Empirical Study of File-System Fragmentation in Mobile Storage Systems. In *Proceedings of the 8th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*. USENIX Association, Denver, CO.
- [17] Yuhun Jun, Jaehyung Park, Jeong-Uk Kang, and Euseong Seo. 2022. Analysis and Mitigation of Patterned Read Collisions in Flash SSDs. *IEEE Access* 10 (2022), 96997–97009.
- [18] Yuhun Jun, Shinhyun Park, Jeong-Uk Kang, Sang-Hoon Kim, and Euseong Seo. 2024. We Ain’t Afraid of No File Fragmentation: Causes

- and Prevention of Its Performance Impact on Modern Flash SSDs. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, Santa Clara, CA, USA, 193–208.
- [19] Saurabh Kadekodi, Vaishnavh Nagarajan, and Gregory R. Ganger. 2018. Geriatrix: Aging what you see and what you don’t see. A file system aging approach for modern storage systems. In *Proceedings of the 2018 USENIX Annual Technical Conference (ATC)*. USENIX Association, Boston, MA, 691–704.
- [20] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. Empowering Storage Systems Research with NVMeVirt: A Comprehensive NVMe Device Emulator. *ACM Transactions on Storage* 19, 4 (2023), 32:1–32:26.
- [21] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. NVMeVirt: A Versatile Software-defined Virtual NVMe Device. In *Proceedings of the 21st USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, Santa Clara, CA, USA.
- [22] Sangjin Lee, Lan Anh Nguyen, Hyeongi Yeo, Sunggon Kim, Sungrae Cho, and Yongseok Son. 2023. A Survey on File Defragmentation Techniques on Modern Storage Systems. In *Proceedings of the 14th International Conference on Information and Communication Technology Convergence (ICTC)*. 785–787.
- [23] Dingbang Liu, Xiangyu Zou, Tao Lu, Philip Shilane, Wen Xia, Wenxuan Huang, Yanqi Pan, and Hao Huang. 2025. Garbage Collection Does Not Only Collect Garbage: Piggybacking-Style Defragmentation for Deduplicated Backup Storage (*EuroSys ’25*).
- [24] Jonggyu Park and Young Ik Eom. 2021. FragPicker: A New Defragmentation Tool for Modern Storage Devices. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*. Virtual, Germany, 280–294.
- [25] Jonggyu Park and Young Ik Eom. 2023. Filesystem Fragmentation on Modern Storage Systems. Association for Computing Machinery.
- [26] Jonggyu Park, Dong Hyun Kang, and Young Ik Eom. 2016. File Defragmentation Scheme for a Log-Structured File System. In *Proceedings of the 7th Asia-Pacific Workshop on Systems (APSys)*. Hong Kong, China.
- [27] Chris Sabol and Ross Stenfort. 2022. Hyperscale Innovation: Flexible Data Placement Mode (FDP). <https://nvmexpress.org/wp-content/uploads/Hyperscale-Innovation-Flexible-Data-Placement-Mode-FDP.pdf>.
- [28] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. 2018. *Operating System Concepts* (10 ed.). Wiley.
- [29] Keith A. Smith and Margo Seltzer. 1996. A Comparison of FFS Disk Allocation Policies. USENIX Association, San Diego, CA. <https://www.usenix.org/conference/usenix-1996-annual-technical-conference/comparison-ffs-disk-allocation-policies>
- [30] Keith A. Smith and Margo I. Seltzer. 1997. File system aging—increasing the relevance of file system benchmarks. In *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*. 203–213.
- [31] Wenxin Wang, Yaqi Li, Liang Shi, and Edwin H.-M. Sha. 2024. Eliminate Critical Fragmentation of F2FS in Mobile Devices with Controller Co-Design. In *Proceedings of the 13th Non-Volatile Memory Systems and Applications Symposium (NVMSA)*.
- [32] Yang Zhou, Fang Wang, Zhan Shi, and Dan Feng. 2024. The Static Allocation is Not a Static: Optimizing SSD Address Allocation Through Boosting Static Policy. *IEEE Transactions on Parallel and Distributed Systems* (2024).
- [33] Guangyu Zhu, Jeongeun Lee, and Yongseok Son. 2022. An efficient and parallel file defragmentation scheme for flash-based SSDs. In *Proceedings of the 37th ACM Symposium on Applied Computing (SAC)*.
- [34] Ningning Zhu, Jiawu Chen, and Tzi-cker Chiueh. 2005. TBBT: Scalable and Accurate Trace Replay for File Server Evaluation. In *Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST)*.